

PRD Update Summary - Tool Transparency & Task Tracking

Date: November 15, 2025

Focus Areas: Tool transparency and task tracking features

Reference: Abacus.AI DeepAgent platform analysis

Overview

This update incorporates comprehensive findings from the Abacus.AI platform analysis into three critical PRD documents, with a focus on tool transparency and task tracking features. The updates provide detailed UI/UX patterns, technical architecture decisions, and implementation recommendations based on proven production patterns.

Key Principle from Analysis

Transparency Over Abstraction - Users need to see exactly what the agent is doing at all times through:

1. Real-time tool usage visibility
2. Comprehensive file operation logging
3. Subtask-level progress tracking
4. Terminal and code output display
5. Clear status indicators throughout

Updated Documents

1. PRD-Real-time Progress Streaming and Transparency.md

Major Additions:

UI/UX Patterns Section

- **Task Card Pattern** - Visual component design for status display
- **Tool Usage Visibility Panel** - Right-side panel showing active tool usage
- **Progress Tracking System** - Bottom status bar format ("Task X, Subtask Y")
- **File Operation Badges** - Badge system for file changes with checkmarks
- **Status Indicators** - Planning → Execution → Completion phases
- **Color Scheme & Typography** - Specific color codes and font guidelines
- **Interactive Elements** - Expandable badges, copy-to-clipboard, hover states
- **Real-time Update Animations** - Smooth transitions and loading states

Technical Architecture Insights

- Real-time Communication Stack (WebSockets, SSE, heartbeat strategy)

- Event Sequencing & Ordering (monotonic sequence numbers)
- Performance Optimizations (batching, virtualization, lazy loading)
- State Management Pattern (PostgreSQL + Redis Pub/Sub)
- Scalability Considerations (horizontal scaling, channel isolation)
- Persistence Strategy (database-first with Redis broadcasting)
- Error Handling & Resilience (auto-reconnect, graceful degradation)

Enhanced Component Architecture

```
<RunProgressPanel>
  <TaskHeader />           // Task card with status
  <MainContent>
    <SubtaskTree />         // Hierarchical tasks
    <CompletionSummary />   // Final operations list
  </MainContent>
  <ToolUsagePanel />        // Right sidebar (conditional)
  <StatusBar />             // Bottom sticky bar
  <ControlButtons />        // Pause/Cancel
</RunProgressPanel>
```

Expanded Event Types

- StatusUpdateEvent (with statusDetail field)
- ToolCallStartEvent / ToolCallEndEvent
- FileOperationEvent (with diff availability)
- StdoutEvent (ANSI-formatted)
- ProgressUpdateEvent (task/subtask counters)

Priority Classification:

- **P0 (MVP):** task-card, tool-usage-panel, status-bar, file-operation-badge
- **P1:** run-progress-tree, subtask-node, event-log, control-buttons
- **P2:** terminal-output, diff-viewer, completion-summary, artifact-list

2. PRD-Intelligent Task Decomposition and Planning.md

Major Additions:

Planning & Execution Flow

- **Multi-Phase Pattern:**
 - Phase 1: Clarification (structured Q&A)
 - Phase 2: Planning (visual status indicators)
 - Phase 3: Execution (real-time visibility)

Task Hierarchy Display Patterns

- Task Card Structure with subtask tree
- Subtask visibility patterns (current, completed, pending, failed)
- Visual checkmarks and progress indicators

Progress Tracking Mechanisms

- Bottom status bar specification ("Task X, Subtask Y")
- Sticky positioning for constant visibility

- Real-time updates as execution progresses

File Operations as Task Evidence

- Badge format: “[Action] ~/path/to/file”
- Operations: Written ✓, Updated ✓, Running, Deleted
- Download buttons on hover
- **Key Learning:** Users trust what they can see

Task Decomposition Granularity Guidelines

Optimal Range: 5-20 subtasks per task, each 10-60 seconds

Good Examples:

- “Initialize Next.js project” (atomic, ~10-30 seconds)
- “Install dependencies via npm” (atomic, visible output)
- “Create homepage component” (atomic, produces artifact)

Avoid - Too Coarse:

- “Build entire application” (too vague)
- “Set up frontend” (too broad)

Avoid - Too Fine:

- “Write import statement” (too granular)
- “Add semicolon to line 42” (unnecessary detail)

User Interaction Points

- **Before Execution:** Review, edit, adjust
- **During Execution:** Pause, cancel, view logs
- **After Completion:** Feedback, download, iterate

Cost & Resource Estimation

- Credits displayed prominently: “Credits Used: 623”
- Show estimates before execution
- Break down by: LLM calls, tool usage, deployment
- No hidden costs

3. PRD-Specialized Tool Ecosystem.md

Major Additions:

Tool Visibility Patterns

Tool Usage Panel Design:

- Right-side panel with automatic appearance
- Header: “Task 1: [task name] > [current subtask]”
- Badge: “🔧 DeepAgent is using [Tool Name]”
- Context breadcrumb for navigation

Tool-Specific Output Display

Web Search:

- Image grid layout

- Thumbnail + title + source URL
- Clickable result cards

Terminal/Command:

- Monospace with ANSI color preservation
- Real-time streaming output
- Syntax highlighting
- Expandable sections

Code Editor:

- Syntax-highlighted display
- Line numbers
- File path header
- Git-style diff view

Data Analysis/Charts:

- Embedded visualizations
- Download buttons
- Data summary tables
- Interactive previews

Tool Execution Status Badges

Progression:

Queued  Running  Succeeded/Failed
[Gray] [Blue] [Green/Red]

Badge Content:

- Tool icon (  )
- Tool name
- Duration
- Status indicator (spinner/checkmark/X)

Tool Output Logging

```
[12:34:56] Tool: web_search
[12:34:56] Input: {"query": "..."}
[12:34:57] Status: Fetching results...
[12:34:59] ✓ Complete: Found 8 relevant results
```

Artifact Management UI

- List view: filename, type, size, timestamp
- Preview button (images, text, JSON)
- Download button (signed URL)
- Delete button (optional)

Multi-Tool Coordination Display

Timeline View:

1.  Web Search (completed) - 2.3s
 Found 8 results
2.  **Data** Analysis (completed) - 5.1s
 Processed 100 rows
3.  Chart Generation (**running**) - 3.2s...

Error Handling Display

- Clear error message (user-friendly)
- Technical details (expandable)
- Retry button for transient failures
- Link to logs
- Suggested actions

Implementation Priority

Must-Have for MVP (All PRDs)

1. Task Card with status display
2. Tool Usage Panel (right sidebar)
3. Status Bar (bottom, sticky)
4. File Operation Badges
5. Real-time status updates
6. Basic output display
7. Artifact list with download

High Priority (Phase 1)

1. Tool-specific renderers
2. Log streaming
3. Pause/Cancel controls
4. Diff viewer
5. Clarification phase UI
6. Plan review interface

Nice-to-Have (Phase 2)

1. Search within logs
 2. Export execution report
 3. Graph visualization
 4. Plan versioning
 5. Interactive charts
 6. Tool analytics
-

Technical Architecture Decisions

Real-time Communication

- **Primary:** WebSockets for bidirectional updates
- **Fallback:** Server-Sent Events (SSE)
- **Heartbeat:** Every 20 seconds
- **Recovery:** Last-Event-ID for reconnection

Event Management

- Monotonic sequence numbers per run
- Atomic sequence assignment via database
- Client tracks last sequence for recovery
- Server replays missed events from DB

Performance

- Batch UI updates with `requestAnimationFrame`
- Virtualized lists for large logs
- Progressive disclosure (collapsed by default)
- Lazy loading of details and artifacts

State Management

- **Server:** PostgreSQL (source of truth) + Redis Pub/Sub
- **Client:** `useReducer` with deduplication
- **Optimistic updates:** For user actions
- **Revalidation:** On reconnect

Scalability

- Separate Redis connections (pub/sub)
 - Channel-per-run isolation
 - Control channel separate from events
 - Horizontal scaling via shared Redis
-

Visual Design Guidelines

Color Palette

```
/* Primary */
--primary: #7C3AED;           /* Purple for actions */

/* Status Colors */
--success: #10B981;           /* Green for completion */
--error: #EF4444;             /* Red for errors */
--warning: #F59E0B;           /* Orange for warnings */
--running: #3B82F6;           /* Blue for active */

/* Neutral */
--bg-primary: #F3F4F6;        /* Light gray background */
--bg-secondary: #E5E7EB;      /* Medium gray */
--border: #D1D5DB;            /* Border gray */
```

Typography

- **Monospace:** Code, file paths, terminal output (Fira Code, Courier)
- **Sans-serif:** UI elements (Inter, System UI)
- **Hierarchy:** h3 for task titles, smaller for metadata

Component Spacing

- Task cards: 16px padding, 8px gap between elements
- Badges: 8px padding, 4px border radius
- Panels: 24px padding, 12px gap
- Status bar: 12px padding, fixed bottom position

ShadCN Components Used

Core Components

- **Badge** - Status and operation badges
- **Card** - Task cards and panels
- **Button** - All actions
- **Accordion** - Expandable subtasks
- **ScrollArea** - Logs and event lists
- **Tooltip** - Hover information
- **Progress** - Progress bars
- **Separator** - Visual divisions

Layout Components

- **Sheet** - Right-side tool panel
 - **Dialog** - Modals for important actions
 - **Tabs** - View switching (code/preview)
 - **Collapsible** - Expandable sections
-

Key Learnings from Reference Platform

1. Transparency Builds Trust

- Users need to see what's happening
- Real-time updates prevent anxiety
- File operations serve as visible proof
- Clear error messages enable debugging

2. Granularity Matters

- Tasks should be 10-60 seconds each
- Too broad: no feedback
- Too fine: noisy and overwhelming
- Optimal: 5-20 subtasks per task

3. User Control is Essential

- Pause/cancel buttons required
- Not just observation - active control
- Feedback mechanisms (👍👎)
- Ability to iterate and refine

4. Cost Transparency

- Show resource consumption clearly
- "Credits Used: 623" format
- No hidden costs
- Estimate before execution

5. Progressive Disclosure

- Start simple (input box)
- Reveal complexity as needed (panels)
- Expandable details (badges)
- Collapsible responses

Next Steps

Immediate Actions

1. **Review updates** with development team
2. **Prioritize components** based on P0/P1/P2 classification
3. **Create UI mockups** using the patterns specified
4. **Design system setup** with color palette and typography

Technical Planning

1. **Architecture review** of real-time communication stack
2. **Database schema** for events and artifacts
3. **Component library** setup with ShadCN
4. **State management** strategy (React Query + useReducer)

Implementation Phases

Phase 1: MVP (2-3 weeks)

- Task cards with status
- Tool usage panel
- Status bar
- File operation badges
- Basic real-time updates

Phase 2: Enhanced Features (2-3 weeks)

- Tool-specific renderers
- Log streaming
- Pause/cancel controls
- Diff viewer

Phase 3: Advanced Features (2-3 weeks)

- Graph visualization
- Search and filtering
- Export functionality
- Analytics dashboard

Files Added/Modified

New Files

- docs/abacus_platform_analysis.md - Comprehensive platform analysis
- docs/PRD/PRD-Real-time Progress Streaming and Transparency.md - Updated
- docs/PRD/PRD-Intelligent Task Decomposition and Planning.md - Updated
- docs/PRD/PRD-Specialized Tool Ecosystem.md - Updated
- docs/PRD/PRD-Multi-LLM Routing and Execution.md - Added to docs
- docs/PRD/PRD-Browser Automation.md - Added to docs
- docs/PRD/PRD-Code Execution Environment.md - Added to docs
- docs/PRD/PRD-File and Artifact Management System.md - Added to docs

Git Commit

```
commit 828d0bd
docs: Update PRDs with Abacus.AI platform analysis findings
9 files changed, 6318 insertions(+)
```

Conclusion

These PRD updates provide a comprehensive foundation for implementing tool transparency and task tracking features based on proven production patterns from Abacus.AI's DeepAgent platform. The detailed UI/UX specifications, technical architecture insights, and implementation priorities ensure the development team has clear guidance for building user-facing features that prioritize transparency and trust.

The focus on **progressive disclosure**, **real-time feedback**, and **user control** creates a framework for an agent platform that users can understand, trust, and effectively use for complex tasks.

Document Version: 1.0

Last Updated: November 15, 2025

Author: DeepAgent Analysis Team